

DATA 266 - HW1 Report

SID4: 6638

Personal Parameters

SID4 = 6638, SEED = 6638, SLICE = 638, HP_ID = 2, CLS_A = 8, CLS_B = 5

HP_ID 2 configuration: hidden layers [64, 32], learning rate 0.003, 30 epochs.

1. What Are Autoregressive Models?

An autoregressive model is a generative model that produces a sequence one element at a time, where each new element is conditioned on all previously generated elements. Formally, the model factors a joint distribution into a product of conditionals:

$$P(x_1, x_2, \dots, x_n) = P(x_1) \cdot P(x_2|x_1) \cdot P(x_3|x_1, x_2) \cdot \dots \cdot P(x_n|x_1, \dots, x_{n-1})$$

At each step, the model predicts the next value given only the past. The key defining property is that generation proceeds left-to-right (or in a fixed ordering), with each output depending solely on prior outputs.

Real-World Examples

GPT and Large Language Models: Models like GPT-4 generate text one token at a time. Each token is predicted based on all preceding tokens in the sequence, making the generation process autoregressive.

WaveNet (Audio Generation): Developed by DeepMind, WaveNet generates raw audio waveforms sample by sample. Each audio sample is conditioned on all previous samples, enabling highly realistic speech synthesis.

PixelCNN / PixelRNN (Image Generation): These models generate images one pixel at a time, scanning from top-left to bottom-right. Each pixel's color value is predicted based on all previously generated pixels.

ARIMA (Time-Series Forecasting): The classical Autoregressive Integrated Moving Average model predicts the next value in a time series as a linear combination of previous values. It is one of the earliest and most widely used autoregressive models.

2. Diabetes Dataset: Neural Network Implementation

The dataset contains 759 samples with 8 pre-normalized features and a binary outcome (diabetes: 0 or 1). The data was split into training (531, 70%), validation (114, 15%), and testing (114, 15%) using random_state = 6638. Two models were trained in each framework: a baseline (lr=0.001) and a modified model per HP_ID=2 (lr=0.003), both with hidden layers [64, 32], Adam optimizer, ReLU activations, sigmoid output, binary cross-entropy loss, and 30 epochs. Each was trained with seeds 6638, 6639, and 6640.

Data Visualization

Figure 1: Correlation Matrix

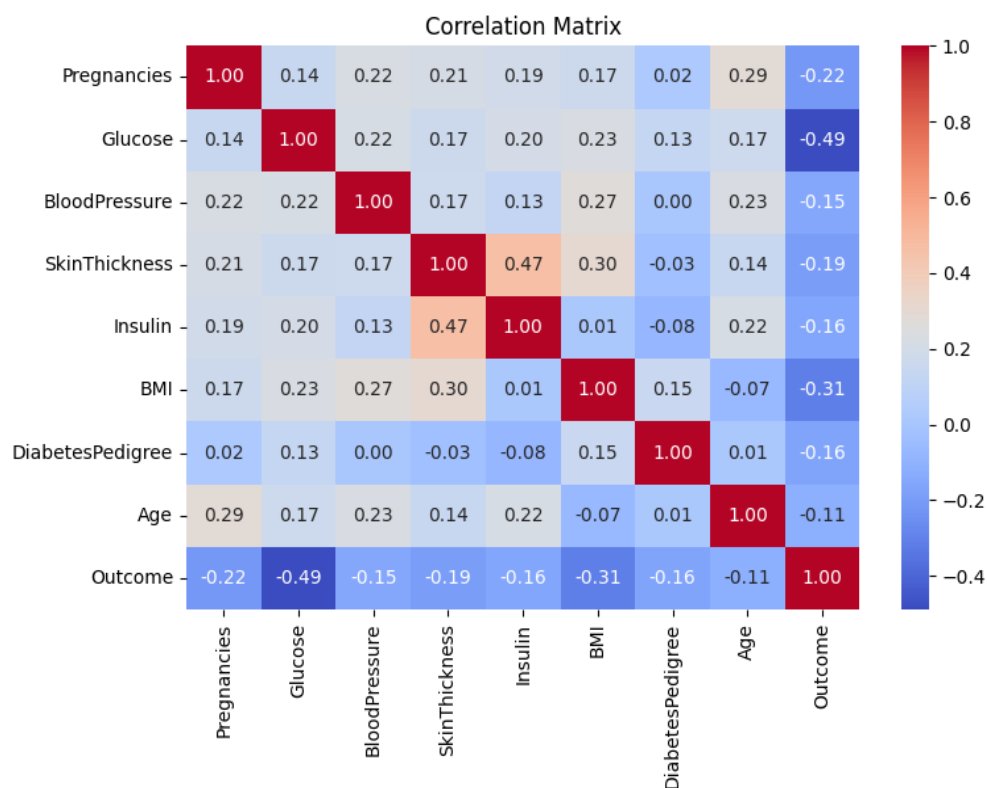
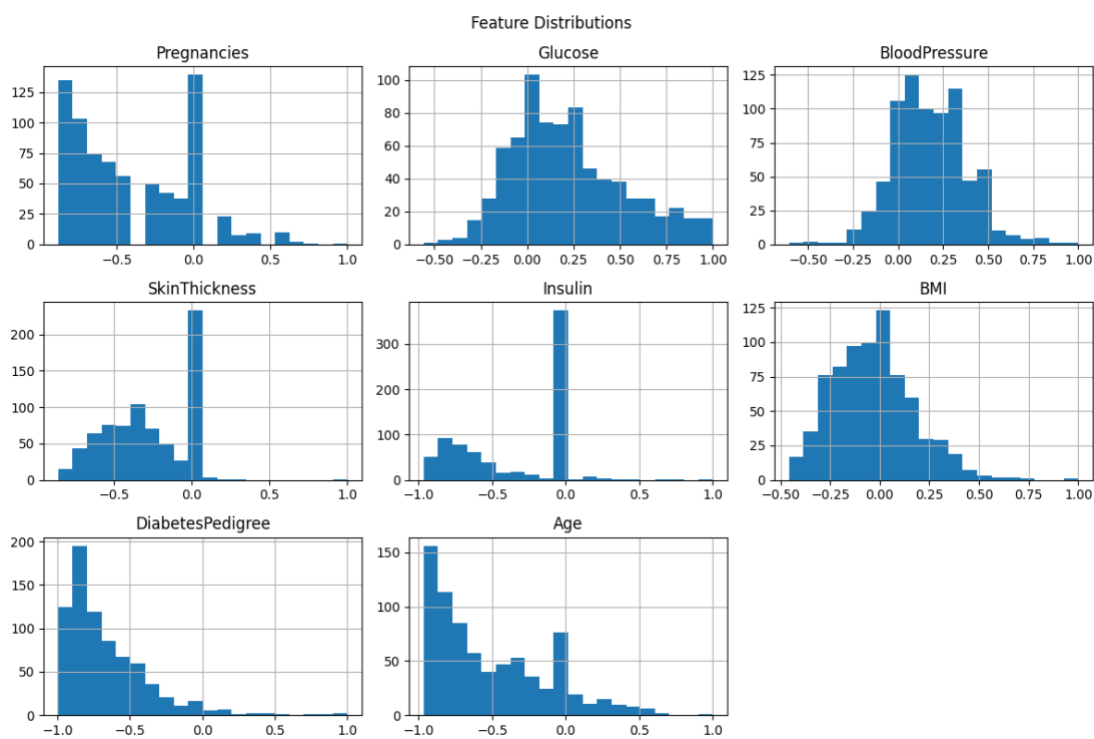


Figure 2: Feature Distributions

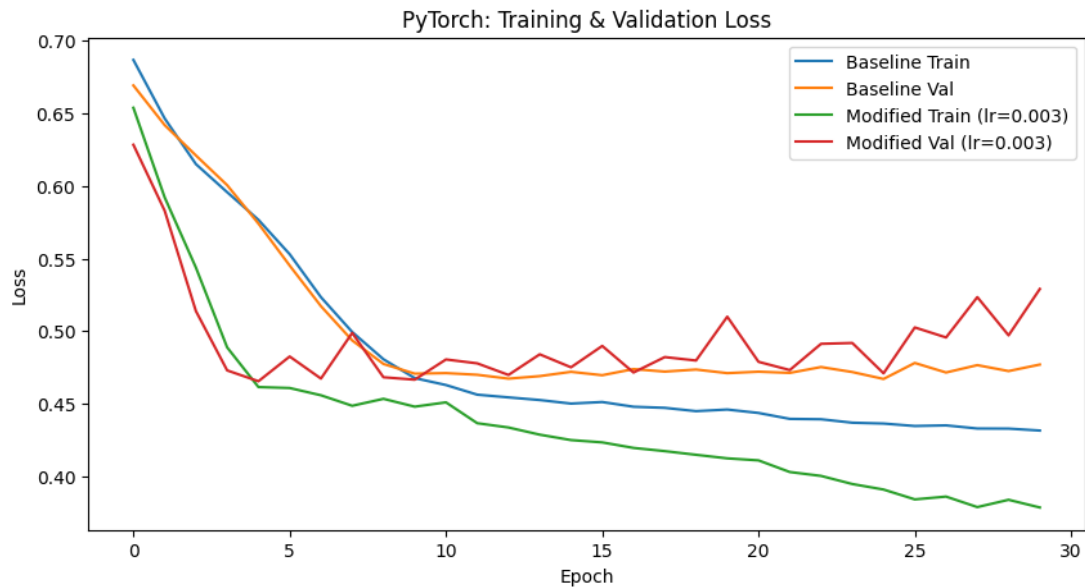


PyTorch Results

Model	Seed 6638	Seed 6639	Seed 6640	Mean	Std
-------	-----------	-----------	-----------	------	-----

Baseline (lr=0.001)	0.7281	0.7719	0.7368	0.7456	0.0189
Modified (lr=0.003)	0.7456	0.7544	0.7456	0.7485	0.0041

Figure 3: PyTorch Training & Validation Loss Curves

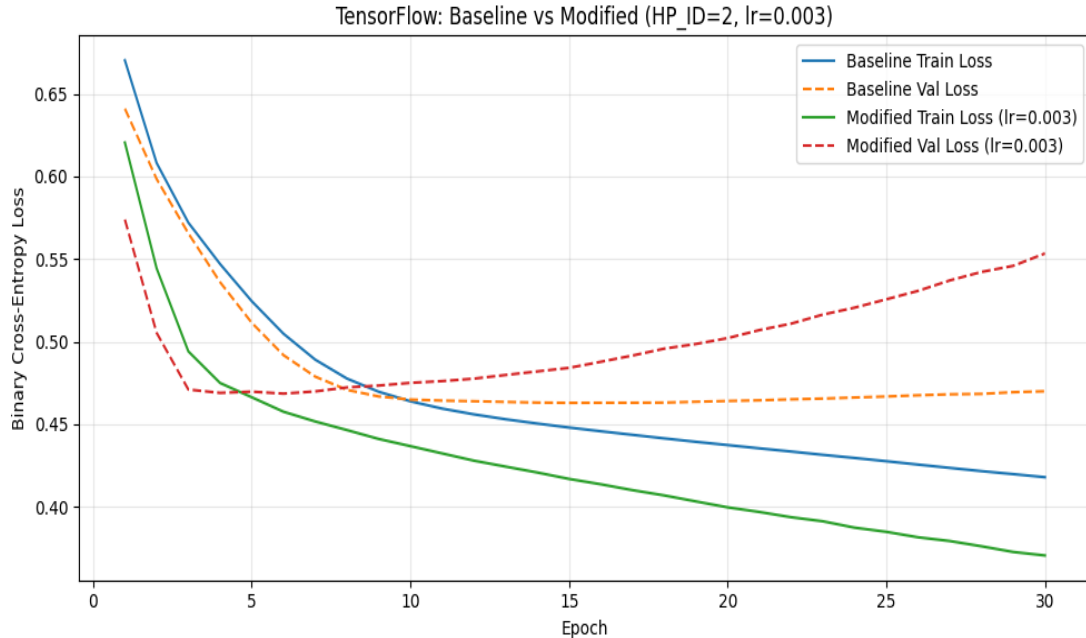


The higher learning rate (0.003) produced a marginal accuracy improvement (0.7485 vs 0.7456) and notably lower variance across seeds (0.0041 vs 0.0189), indicating more stable convergence. The loss curves show the modified model's training loss decreasing faster in early epochs due to the larger step size, reaching a lower plateau sooner. However, the modified model's validation loss begins to diverge from its training loss after epoch 10, showing early signs of overfitting. The baseline's validation loss tracks its training loss more closely. Despite this, the modified model achieves slightly better overall test accuracy. The modified model is slightly better overall.

TensorFlow Results

Model	Seed 6638	Seed 6639	Seed 6640	Mean	Std
Baseline (lr=0.001)	0.7368	0.7456	0.7368	0.7398	0.0041
Modified (lr=0.003)	0.7193	0.7018	0.7193	0.7135	0.0083

Figure 4: TensorFlow Training & Validation Loss Curves



In TensorFlow, the baseline (lr=0.001) outperformed the modified model (0.7398 vs 0.7135). The loss curves clearly show the problem: while the modified model's training loss drops faster and lower than the baseline's, its validation loss rises sharply after epoch 8, indicating overfitting. The baseline's validation loss remains flat and stable throughout training. The higher learning rate caused the model to memorize training data without generalizing to the validation set.

Cross-Framework Comparison

Model	PyTorch	TensorFlow
Baseline (lr=0.001)	0.7456 ± 0.0189	0.7398 ± 0.0041
Modified (lr=0.003)	0.7485 ± 0.0041	0.7135 ± 0.0083

The effect of tripling the learning rate was framework-dependent. In PyTorch, lr=0.003 slightly improved accuracy and substantially reduced variance. In TensorFlow, lr=0.003 hurt accuracy by about 2.6 percentage points and increased variance, with clear overfitting visible in the loss curves. Although both frameworks use Adam with batch_size=32, their weight initialization schemes and internal numerical implementations differ slightly, changing the loss landscape enough that the same learning rate adjustment helps in one framework but hurts in the other.

3. CUDA Matrix Multiplication

Blocks and Threads

The CUDA kernel uses a 2D grid of 2D thread blocks. Each block contains $16 \times 16 = 256$ threads. The grid is dimensioned as $\text{ceil}(N/16) \times \text{ceil}(N/16)$ blocks, ensuring every element of the $N \times N$ output matrix is assigned exactly one thread. Each thread computes its (row, col) position using blockIdx and threadIdx, then loops over the shared dimension k to compute one dot product: $C[\text{row}][\text{col}] = \sum A[\text{row}][k] \cdot B[k][\text{col}]$.

Timing Results

Matrix Size	CPU (ms)	GPU Kernel (ms)	H2D+D2H (ms)	Speedup
256	54.43	0.35	0.46	67.13x
1024	7,292.28	8.38	5.51	524.95x

4096	1,762,997.95	315.76	74.26	4,520.38x
------	--------------	--------	-------	-----------

Profiler used: nvprof on Google Colab T4 GPU. The profiler output confirms that for $N=1024$, the kernel accounts for 74.5% of GPU time while memory transfers account for the remaining 25.5%.

Crossover Analysis

The GPU is beneficial at all three sizes tested, with the crossover likely below $N=256$. The speedup increases dramatically with matrix size because computation scales as $O(N^3)$ while memory transfer scales as $O(N^2)$. At small N , kernel launch latency and memory transfer overhead consume a larger fraction of total GPU time, reducing the speedup. As N grows, the massively parallel $O(N^3)$ computation dominates, and the GPU's thousands of cores provide increasingly dramatic speedups over the single-threaded CPU. The crossover is not at size zero because every GPU operation carries fixed overhead: driver initialization, kernel launch, and PCIe bus transfer latency, all of which are constant regardless of problem size.